

Reviewer Pack — Minimal Symplectic Leaves Number Bounds Tree-Like Resolution...

Entry 53e9213f6fb0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 19:13:51 UTC

Minimal Symplectic Leaves Number Bounds Tree-Like Resolution Width

Entry ID: 53e9213f6fb0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 19:13:51 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Boolean Function Complexity: Tree-like Resolution Proof Complexity

Statement:

For a given Boolean function f , the minimal number of symplectic leaves in its associated symplectic leaf system L_f is polynomially related to the tree-like resolution width $w_t(f)$ such that $|L_f| \leq c * w_t(f)^2$ for some constant c .

Rationale (proposer's reasoning):

Symplectic geometry provides a way to encode Boolean functions through their associated symplectic leaf systems. If the minimal number of leaves is related to the resolution width, it suggests that the structure of the symplectic leaves can provide insights into proof complexity.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1b643d31c330eeb7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the computed correlation between the minimal number of leaves $|L_f|$ and $w_t(f)^2$ across 30 seeds is significantly positive ($p\text{-value} < 0.05$) and the correlation coefficient exceeds 0.7, otherwise it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "symplectic geometry" AND "Boolean function complexity" AND tree-like resolution - "minimal symplectic leaves number bounds" AND tree-like resolution width - "symplectic leaf system" AND polynomial relation Boolean function complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tree_like_resolution_width(f):
        n = int(math.log2(len(f)))
        clauses = []
        for i in range(n):
            clause = []
            for j in range(i + 1):
                if f[2**(i - j) ^ 2**j] != f[2**(i - j)]:
                    clause.append(j)
            clauses.append(clause)
        count = 0
        while len(clauses) > 1:
            new_clauses = []
            for i in range(0, len(clauses), 2):
                if i + 1 < len(clauses):
                    new_clause = set(clauses[i]) | set(clauses[i + 1])
                    new_clauses.append(new_clause)
                else:
                    new_clauses.append(clauses[i])
            clauses = new_clauses
            count += 1
        return len(clauses) - count

    def symplectic_leaves(f):
```

```

    n = int(math.log2(len(f)))
    leaves = []
    for i in range(2**n):
        if all(f[i ^ (1 << j)] == f[i] for j in range(n)):
            leaves.append(i)
    return len(leaves)

instances_tested = 0
total_leaves = 0
total_widths_squared = 0

for _ in range(30):
    n = random.randint(5, 40)
    f = generate_boolean_function(n)
    w_t_f = tree_like_resolution_width(f)
    leaves = symplectic_leaves(f)

    instances_tested += 1
    total_leaves += leaves
    total_widths_squared += w_t_f ** 2

metric_value = total_leaves / instances_tested
n_max = 40
conjecture_holds = False
counterexample = ""

return {
    "metric_name": "Symplectic Leaves Number",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the required correlation analysis could not be completed. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16328
2	propose	ollama_remote	glm4:latest	0	0	15829
3	propose	ollama_remote	glm4:latest	0	0	26553
4	preregistration	ollama_remote	glm4:latest	0	0	9914
5	novelty	ollama_remote	glm4:latest	0	0	14429
6	novelty	ollama_remote	glm4:latest	0	0	11121
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16885
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16596
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12130
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9405
11	judge	ollama_remote	glm4:latest	0	0	35071

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 184262 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/53e9213f6fb0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/53e9213f6fb0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/53e9213f6fb0.tar.gz (if generated)